Security Report: Justification for Enforcing Elasticsearch Authentication

1. Executive Summary

This report outlines the critical importance of the recent security hardening implemented in the `mx-chain-es-indexer-go-NewArc` repository. Specifically, the enforcement of `ELASTIC_USERNAME` and `ELASTIC_PASSWORD` directly at the application code level represents a shift to a **"Secure by Default"** and **"Fail-Closed"** architecture.

By modifying the application to panic and refuse execution when credentials are not provided (unless explicitly bypassed for local development), the system actively prevents catastrophic data loss, manipulation, and infrastructure compromise in production environments.

2. The Threat Landscape for Unsecured Elasticsearch

Elasticsearch instances exposed without authentication (`xpack.security.enabled=false`) are among the most frequently targeted infrastructure components on the internet. Because Elasticsearch communicates via standard HTTP REST APIs, an unsecured instance grants **full read, write, and administrative access** to anyone who can reach the IP address.

Leaving the indexer's database unauthenticated exposes the network to three primary critical-severity threats:

A. Automated Ransomware and Data Deletion (The "Meow" Attack)

- **Threat:** Automated botnets continually scan the internet (via services like Shodan) for exposed port `9200` .
- **Impact:** Within hours of exposure, bots will systematically send `DELETE /*` commands to wipe all indices, often replacing them with a single index demanding a cryptocurrency ransom.
- **Consequence:** While blockchain data is inherently public and recoverable, re-indexing millions of blocks and transactions from scratch takes days or weeks. During this recovery period, any decentralized applications (DApps), block explorers, or analytical tools relying on the indexer will experience complete downtime.

B. Data Integrity Compromise and Manipulation

- **Threat:** Unauthenticated access means write-access is public. Malicious actors can use standard `POST` and `PUT` requests to inject fabricated data into the indices.
- **Impact:** Attackers can artificially inflate account balances, fabricate transaction histories, or alter smart contract execution results within the database.
- **Consequence:** Front-end interfaces relying on the indexer will display this manipulated data. This can cause severe reputational damage, trigger false alerts, and potentially trick end-users into making financial decisions based on spoofed information.

C. Infrastructure Hijacking and Denial of Service (DoS)

- **Threat:** The Elasticsearch Query DSL is extremely powerful. Unauthenticated users can craft intentionally complex, deeply nested aggregation queries.
- **Impact:** These "malicious queries" force the database engine to consume all available CPU and JVM heap memory, resulting in OutOfMemory (OOM) crashes and permanent Denial of Service until the

node is manually restarted. Furthermore, unpatched Elasticsearch instances are frequently targeted for Remote Code Execution (RCE) to install cryptominers on the host servers.

3. Why Code-Level Enforcement Was Necessary

Prior to this implementation, the responsibility of securing Elasticsearch fell entirely on external bash scripts (like `script.sh`) or DevOps configurations (like `docker-compose.yml`). This is an anti-pattern known as relying solely on "perimeter defense."

Enforcing credentials directly in `main.go` and `config.go` provides several crucial architectural benefits:

1. **Defense in Depth:** The application itself now acts as a secondary layer of security. Even if a DevOps engineer makes a mistake and deploys a `docker-compose.yml` with security disabled, the Go application will refuse to interface with it or start up without credentials.
2. **Preventing Accidental Deployments:** Human error is the leading cause of exposed databases. The application now "Fails-Closed"—crashing immediately with a clear error message about missing credentials—rather than silently succeeding and leaving the database vulnerable.
3. **Controlled Local Development:** By introducing the `allow-insecure-no-auth-dev` flag, the development team maintains the ability to run rapid, local testing without the friction of managing passwords, but requires an explicit, conscious action to bypass security. This flag is never set in production templates.

4. Hardening the Integration Test Suite (`utils.go`)

As a direct consequence of enforcing a "Secure by Default" architecture, the Elasticsearch instance spawned for continuous integration and local testing (via `scripts/script.sh`) now correctly boots with X-Pack security enabled.

This introduced a necessary regression where the integration test suite failed entirely with `401 Unauthorized` errors. The tests were previously designed to instantiate their database clients anonymously. To resolve this and ensure the tests accurately reflect production-like conditions, the testing infrastructure in `integrationtests/utils.go` was updated:

1. `createESClient` Authentication Injection

The client constructor was modified to dynamically read `ELASTIC_USERNAME` and `ELASTIC_PASSWORD` from the testing environment and inject them into the `elasticsearch.Config` struct.

Before:

```
func createESClient(url string) (elasticproc.DatabaseClientHandler, error) {
    return client.NewElasticClient(elasticsearch.Config{
        Addresses: []string{url},
        Logger:    &logging.CustomLogger{},
    })
}
```

After:

```
func createESClient(url string) (elasticproc.DatabaseClientHandler, error) {
    username := os.Getenv("ELASTIC_USERNAME")
    if username == "" {
```

```
        username = "elastic"
    }
    password := os.Getenv("ELASTIC_PASSWORD")

    return client.NewElasticClient(elasticsearch.Config{
        Addresses: []string{url},
        Username:  username,
        Password:  password,
        Logger:    &logging.CustomLogger{},
    })
}
```

2. `getIndexMappings` Basic Auth

Standard HTTP GET requests used for mapping verification were upgraded to use `http.NewRequest` with `SetBasicAuth()`, injecting the required headers for authentication.

Before:

```
func getIndexMappings(index string) (string, error) {
    u, _ := url.Parse(esURL)
    u.Path = path.Join(u.Path, index, "_mappings")
    res, err := http.Get(u.String())
    if err != nil {
        return "", err
    }
    // ...
```

After:

```
func getIndexMappings(index string) (string, error) {
    u, _ := url.Parse(esURL)
    u.Path = path.Join(u.Path, index, "_mappings")

    req, err := http.NewRequest(http.MethodGet, u.String(), nil)
    if err != nil {
        return "", err
    }

    username := os.Getenv("ELASTIC_USERNAME")
    if username == "" {
        username = "elastic"
    }
    password := os.Getenv("ELASTIC_PASSWORD")
    if password != "" {
        req.SetBasicAuth(username, password)
    }

    res, err := http.DefaultClient.Do(req)
    if err != nil {
        return "", err
```

```
    }
    // ...
```

By propagating these security requirements into the test suite itself, we guarantee that all CI/CD pipelines validate the codebase against a strictly authenticated database, preventing any future regressions that might accidentally assume an unsecured backend.

5. Conclusion

The implementation of mandatory `ELASTIC_USERNAME` and `ELASTIC_PASSWORD` checks within the application code is not merely a bureaucratic compliance step; it is a fundamental infrastructure safeguard.

By forcing the application to reject unauthenticated states, the `NewArc` repository has successfully mitigated the highest-probability vectors for data loss and system compromise. This hardening is a mandatory prerequisite for any production-grade deployment of the MultiversX elastic indexer.